



EMI305 · PRODUCCIÓN DE SOFTWARE · SISTEMAS CIBERFÍSICOS

Semana 2 — Modelos, metamodelos, DSL y Desarrollo Dirigido por Modelos

¿Por qué modelamos? · Jerarquía de modelado · ¿Basta UML? · Sintaxis abstracta, concreta y semántica · MDD como cadena de transformaciones.

Magíster en Ingeniería Informática · UFRO

EMI305 · Producción de Software · 2026



INSTITUCIÓN ACREDITADA
6 AÑOS
EN TODAS LAS ÁREAS
• GESTIÓN INSTITUCIONAL • DOCENCIA DE PREGRADO
• DOCENCIA DE POSTGRADO • INVESTIGACIÓN
• VINCULACIÓN CON EL MEDIO

Qué vamos a recorrer

01 ¿Qué es un modelo y para qué sirve?

02 Niveles de modelado (la pirámide)

03 Relación modelo ↔ paradigma de modelado

04 ¿Basta UML? → los DSL

05 Sintaxis abstracta · concreta · semántica

06 Ejemplos de DSL en otras industrias

07 MDD: el ciclo de vida como cadena de transformaciones

08 Aplicación al caso SVIF

09 En síntesis · Referencias

¿Qué es un modelo y para qué sirve?

«Un modelo es una **representación de la realidad para algún propósito definido**» — Pidd (2003).

Ejemplo de clase: distintos **mapas de una ciudad** (transporte público, ciclovías, ruidos) resaltan aspectos distintos según el propósito.

Un modelo es útil para:

DOCUMENTAR

RAZONAR

COMUNICAR

Un modelo útil es (Selic, 2003):

CALIDAD
Abstracto

CALIDAD
Comprensible

CALIDAD
Preciso · Predictivo · Barato

Relaciones clave



SVIF: antes de escribir una línea de C++, el sistema queda representado por modelos que permiten razonar sobre objetivos en conflicto, documentar la arquitectura y comunicar decisiones.

Niveles de modelado: ¿quién define las reglas?

Analogía cartográfica y su equivalente en software:



NIVEL (MAPA)	ESPACIO VISUAL	ESPACIO TEXTUAL
Ciudad (realidad)	Sistema	Sistema
Mapa	Modelo	Código fuente
Leyenda	Metamodelo (p. ej., UML)	Sintaxis
Cartografía	Metametamodelo (MOF)	Metalenguaje (EBNF)

¿Basta UML? → los DSL

UML describe muy bien clases, objetos, componentes e interacciones, pero dominios como los CPS incorporan **conceptos muy específicos** (mecanismos físicos, hilos periódicos, recursos de hardware).

Extender UML es posible pero incrementa la complejidad. La alternativa: **crear un lenguaje propio**.

«Un **DSL** (Domain-Specific Language) es un lenguaje bien definido diseñado para expresar soluciones utilizando conceptos especializados de un dominio particular» — adaptado de Fowler & Parsons (2010); Mernik, Heering & Sloane (2005).

REGLA PRÁCTICA

¿Cuándo construir un DSL?

El DSL debe aportar **suficiente valor al dominio** para justificar construirlo y mantenerlo.

Ventajas

- Conceptos expresivos
- Modelos comprensibles para expertos del dominio
- Menos complejidad y errores

Desventajas

- Curva de aprendizaje
- Falta de estándares/herramientas
- Costo de evolución

Todo DSL tiene tres elementos

1 · SINTAXIS ABSTRACTA

Metamodelo

Conceptos, relaciones y reglas de buena formación.

`CPCComponent`, `OnIntervalAction`,
`OnDemandAction`, `SWResource`,
`HWResource`, `MessageSender/Receiver`

2 · SINTAXIS CONCRETA

Notación

Representación gráfica o textual.

Biblioteca de figuras para `diagrams.net` (*scratchpad PIM-DSL*).

3 · SEMÁNTICA

Significado

Cómo interpretar los conceptos; se define traduciéndolos a un lenguaje destino conocido.

`OnIntervalAction`
→ hilo periódico en C++

Ejemplos de DSL en otras industrias:

`SQL` bases de datos `HTML` documentos web `TERRAFORM` infraestructura `SIMULINK` control `BPMN` procesos de negocio

MDD: el ciclo de vida como cadena de transformaciones

Bézivin (2005) propone ver **el ciclo de vida del software como una cadena de transformaciones de modelos**.

Dos artefactos principales:

MODELOS

TRANSFORMACIONES

No es una idea nueva: diagrama de clases → generación de código → Java y código fuente → compilación → ejecutable ya son transformaciones entrada-proceso-salida.



Cada transformación preserva trazabilidad (

`id_cim_parent`) e incorpora decisiones del siguiente nivel. SVIF: CIM (objetivos, softgoals) → PIM (períodos, dependum) → PSM (ESP32 + MQTT, tipos concretos) → Code (credenciales, política). "agente": lo aplicó directamente el diseñador. La herramienta oficial `aomdd4cps` automatiza CIM → PIM y PIM → PSM con XSLT, y PSM → Code con Python.

En síntesis

Modelamos para **razonar barato** y sin ambigüedad.

Las reglas del modelado viven en el **metamodelo** (M2), con un **metametamodelo** (M3) que las autodescribe.

Cuando UML no expresa bien el dominio, se construye un **DSL** (metamodelo + notación + semántica).

MDD encadena transformaciones automáticas de modelos hasta llegar al producto.

*Próxima parada (Semana 3): aplicar la orientación a agentes (**iStar 2.0**) como CIM y construir el **DSL PIM** puente hacia el código CPS.*

Referencias clave: Aßmann, Zschaler & Wagner (2006); Bézivin (2005); Fowler & Parsons (2010); Mernik, Heering & Sloane (2005); Pidd (2003); Selic (2003).